CHAPTER 23

Structured Output and JSON Contracts

In this chapter

Introduction: The Parse Errors Went to Zero and the Incidents Did Not . . . 3
23.1 What Constrained Decoding Actually Is . . . 4
23.2 Two Features, and What Each One Constrains . . . 6
23.3 JSON Outputs: output_config.format . . . 6
23.4 Strict Tool Use: strict: true . . . 7
23.5 The Supported Subset of JSON Schema . . . 8
23.6 The Three Limits You Can Count, and the Ones You Cannot . . . 9
23.7 Four Things the Schema Does Not Promise . . . 10
23.8 Incompatibilities, and What Still Caches . . . 10
23.9 Two Caches, Two Keys . . . 11
23.10 Where the Feature Is Available . . . 12
23.11 Migrating Chapter 22's Envelope . . . 12
23.12 Visual Analogies . . . 13
23.13 Architecture: One Contract, Two Emitters, One Reader . . . 14
23.14 Examples . . . 15
23.15 Code Walkthrough: The Schema Reader That Refuses to Guess . . . 15
23.16 Mini Project: The Schema Compiler and Contract Harness . . . 16
23.17 Panel A — The Contract, Read Before It Is Sent . . . 17
23.18 Panel B — Seventeen Rejections, No API Key . . . 17
23.19 Panel C — Conformance Is Not Correctness . . . 18
23.20 Panel D — The Enum That Was Never in the Enum . . . 19
23.21 Panel E — The Two Paths With No Grammar . . . 20
23.22 Panel F — Two Caches, Two Keys, Priced . . . 21
23.23 Evaluation and Deployment . . . 22
23.24 Hands-on Labs . . . 22
23.25 Lab Solutions . . . 22
23.26 Best Practices . . . 23
23.27 Common Mistakes . . . 25
23.28 Performance Considerations . . . 27

23.29 Security Considerations 28
23.30 Real Production Examples 28
23.31 Certification Notes 29
23.32 Twenty Practice Questions 30
23.33 Answers and Explanations 31
23.34 Ten Practical Exercises 32
23.35 Five Challenge Labs 32
23.36 Ten Interview Questions 32
23.37 Summary 33
23.38 Glossary 34
23.39 Further Reading 35

NOTE

Learning Objectives

By the end of this chapter you will be able to:

1. Explain what **constrained decoding** is — a restriction on the token distribution at sampling time, not a validator that runs afterwards — and where it sits in the machinery of Chapter 3 and Chapter 4.
2. Write both documented forms of the feature from one source of truth: `output_config.format` for JSON outputs and `strict: true` for tool use.
3. Stay inside the supported JSON Schema subset, and say precisely which keywords are outside it, which the SDKs silently rewrite, and which are documented neither way.
4. Enforce the three explicit complexity limits — **20 strict tools, 24 optional parameters, 16 union-typed parameters** — before a request leaves your process.
5. Handle the four cases where the schema is not a promise: `stop_reason: "refusal"`, `stop_reason: "max_tokens"`, enum capitalisation, and property ordering.
6. Recognise the incompatibilities — citations returns a 400, prefilling is unsupported — and know what you must fall back to.
7. Distinguish a response that **conforms** from a response that is **correct**, and measure the two separately.
8. Price a schema: the grammar cache, the prompt cache, and the edit that is free for one and ruinous for the other.

NOTE

Prerequisites

Chapter 3 for softmax and for what "the next-token distribution" means, because that distribution is exactly what a grammar edits. **Chapter 10** for the capability triage and for structured outputs as *the limit that moved*. **Chapter 16** for the Messages API request shape, for typed content blocks, and above all for `stop_reason` as the control-flow branch. **Chapter 19** for cache multipliers and what a token costs. **Chapter 20** for the cacheable prefix and for prompt parts with a kind and a stability. **Chapter 22** for the output envelope this chapter replaces, and for the parser that is still needed where nothing is enforced.

Tool design in the round — naming, descriptions, affordances, error contracts — is Part 6, Chapters 31 to 35. This chapter is about the *schema* as a contract, not about what makes a good tool. Prompt injection through schema fields is Chapter 58.

Introduction: The Parse Errors Went to Zero and the Incidents Did Not

A support-triage service had a metric on a dashboard called `json_parse_failures`. It was the metric everybody watched, because it was the one that woke people up: the classifier returned a JSON object describing a ticket, a downstream router acted on it, and when the object did not parse the ticket went nowhere until somebody noticed.

The team migrated to structured outputs over a weekend. They kept the same model, the same prompt, the same examples. They added twenty lines: a JSON Schema and an `output_config.format` block. On Monday morning `json_parse_failures` read zero, and it has read zero every day since.

The incident rate did not move.

It is worth being precise about what happened, because the shape of it recurs in every migration of this kind. Section 23.19 reproduces the whole thing offline against twenty-four BUILT responses and a gold key. Before the migration, the pipeline delivered a usable answer for **20 of 24** tickets — two responses were not valid JSON at all, one returned `"0.9"` where a number was required, one omitted a required field. After the migration it delivered a usable answer for **24 of 24**. That is the win, and it is a real one.

But of the twenty-four answers, **15 were the right answer** — the right category and the right severity — and that number was **15 before the migration and 15 after it**. The grammar recovered every failure that was a *formatting* failure. It recovered none that was a *judgement* failure, and it was never going to, because a grammar constrains the shape of a sentence and has no opinion about whether the sentence is true.

End to end, correctness went from **12 of 24 (50.0%)** to **15 of 24 (62.5%)**. The remaining 37.5 per cent is untouched, and the dashboard that used to show it now shows zero.

There are two smaller surprises in the same measurement, and both are documented behaviour rather than bugs. **Three of the twenty-four governed responses came back with a category the schema does not contain** — `"feature Request"` where the enum says `"feature request"` — because structured outputs explicitly do not guarantee the capitalisation of enum values. A team that compares with `==` therefore throws away three answers the model got right, taking correctness back down from 62.5 per cent to 50.0 per cent and undoing the entire benefit of the migration. And the six responses that used to carry a helpful undeclared `priority_score` field now cannot: `additionalProperties` must be `false`, so the field the analytics team had quietly started reading simply stopped arriving.

This chapter is about the promise Chapter 22 could not make. It is a real promise, it is narrower than it first appears, and the two things it does not cover are the two things that matter most when something goes wrong.

23.1 What Constrained Decoding Actually Is

Chapter 3 ended a forward pass with a softmax: a vector of logits, one per token in the vocabulary, turned into a probability distribution, from which one token is sampled. Chapter 4 showed that this happens once per output token, each time conditioned on everything generated so far. Nothing in that loop knows what JSON is.

Constrained decoding — the documentation calls the tool-use form **grammar-constrained sampling** — inserts one step into that loop. Your JSON Schema is compiled into a formal grammar. At every decode step the grammar is consulted for the set of tokens that could legally continue the output so far, and every token outside that set has its probability driven to zero before sampling. The model still chooses; it simply cannot choose a token that would make the document invalid.

DEFINITION

Constrained decoding

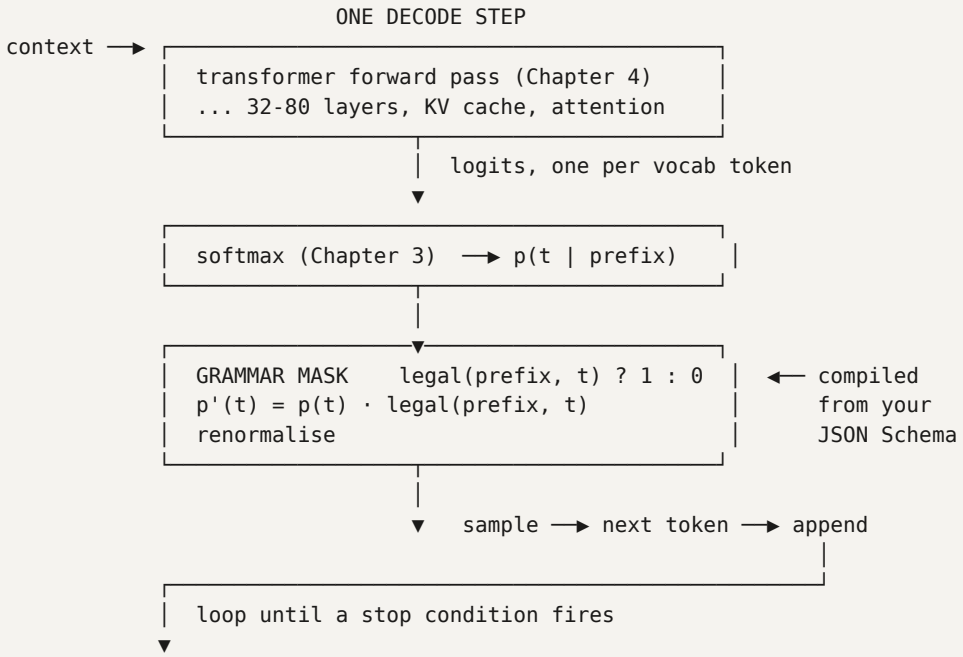
A restriction applied to the next-token distribution at each decode step, so that only tokens permitted by a compiled grammar can be sampled. It is a property of *generation*, not a check performed on the finished text. There is no retry, no repair and no second pass, because an invalid document is never produced in the first place.

The distinction between this and post-hoc validation is the whole chapter, so it is worth stating three ways.

Simply. A validator is a bouncer at the door who turns people away. A grammar is a corridor with no wrong doors in it.

Technically. Post-hoc validation is a predicate applied to a complete string, `valid(s) -> bool`, evaluated after the model has spent its output tokens. Constrained decoding is a mask applied to the distribution, `p'(t) = p(t) · legal(prefix, t)`, evaluated before each token is spent. The first can only reject; the second can only produce.

In consequences. Because the constraint is applied during generation, there are no retries to budget for, no exponential backoff to tune, and no half-finished object to reason about. But because it is applied during generation, it can only constrain *what the tokens are*, never *what they mean*. A grammar that requires a `severity` field of `"S1"` to `"S4"` will get one of those four strings every time. It has no way to know that this ticket was an S1.



The mask is the ONLY addition. Everything above it is Chapter 4 unchanged; everything below it is Chapter 16 unchanged. This is why a refusal or a max_tokens cut-off escapes the schema: both of them are stop conditions, and a stopped loop has no obligation to have finished a legal document.

Chapter 10 catalogued five structural limits of language models and then, honestly, reported that one of them had moved: exact-format output used to be a matter of prompting and hope, and constrained

decoding turned it into a platform guarantee. This chapter is the detail of that guarantee — including the documented exceptions, which Chapter 10 flagged and deferred to here.

The thread to carry: **a grammar constrains the shape of an answer and says nothing about its truth.**

23.2 Two Features, and What Each One Constrains

The documentation describes structured outputs as *two complementary features*, and the distinction between them is not cosmetic. They constrain different channels of the same response.

	JSON outputs	Strict tool use
Request field	`output_config.format`	`strict: true` on a tool
Constrains	what Claude **says** — the text block	how Claude **calls you** — the `tool_use` input
Appears in	`content[].type == "text"`	`content[].type == "tool_use"`, in `input`
Also guarantees	valid JSON matching your schema	the tool `name` is always a real one
Schema field	`format.schema`	`input_schema`

They may be used independently or together in the same request, and the documentation gives a worked example of exactly that: strict tools so the flight search is called with a real date, plus a JSON output format so the final summary comes back as an object. In an agentic loop that is the normal combination — Chapter 18 established that a tool-use loop is one assistant turn, and within that turn the two features govern two different kinds of content block.

One sentence in the documentation is easy to skim past and worth pinning to the wall:

> *Grammars apply only to Claude's direct output, not to tool use calls, tool results, or thinking tags (when using thinking). Grammar state resets between sections, allowing Claude to think freely while still producing structured output in the final response.*

So the JSON grammar does not leak into the thinking block. Chapter 18's thinking content is generated unconstrained, the grammar state resets, and the final text block is constrained. This is the right design — a model forced to think in JSON would think worse — and it is also the reason the two features exist separately rather than as one flag.

23.3 JSON Outputs: `output_config.format`

The request shape is small. `output_config` is a top-level field alongside `model`, `max_tokens` and `messages`; `format` is an object with `type: "json_schema"` and a `schema`.

```
body = {
    "model": "claude-opus-5",
    "max_tokens": 1024,
    "messages": [{"role": "user", "content": "Extract the contact..."}],
    "output_config": {
        "format": {
            "type": "json_schema",
```

```
            "schema": {
                "type": "object",
                "properties": {
                    "name": {"type": "string"},
                    "email": {"type": "string"},
                    "plan_interest": {"type": "string"},
                    "demo_requested": {"type": "boolean"},
                },
                "required": ["name", "email", "plan_interest",
                             "demo_requested"],
                "additionalProperties": False,
            },
        }
    },
}
```

The response is an ordinary Messages response. The JSON arrives as the text of a text content block, which means Chapter 16's rule applies unchanged: it is `response.content`, an array of typed blocks, and `content[0].text` is still the wrong way to reach it. With thinking enabled — which on Fable 5, Opus 5 and Sonnet 5 is the default — block zero is a thinking block and your JSON is somewhere after it.

Two details of history matter if you are reading older code. The parameter used to be top-level `output_format` behind the beta header `structured-outputs-2025-11-13`; it is now `output_config.format` with no header, and the documentation says the old shape keeps working "for a transition period". The Python SDK's `client.messages.parse()` still accepts `output_format=` as a convenience and translates it internally; other SDKs require `output_config` directly.

> WARNING
>
> **The SDK is quietly rewriting your schema**
>
> The Python, TypeScript, Ruby and PHP SDKs — and the C# and Go SDKs when the schema is derived from a native type — transform your schema before sending it. They **remove** unsupported constraints such as `minimum`, `maximum`, `minLength` and `maxLength`; **rewrite the description** to carry the constraint in prose ("Must be at least 100"); **add** `additionalProperties: false` to every object; **filter string formats** to the supported list; and then **validate the response against your original schema**, constraints included.
>
> This is a good design and it is also a trap for anyone reasoning about cost, caching or reproducibility, because the bytes on the wire are not the bytes you wrote. A `minimum: 100` becomes a description string that the model may or may not honour, enforced afterwards by the SDK rather than by the grammar. If you need to know exactly what was sent — and if you are pricing a cached prefix, you do — build the schema yourself or, in TypeScript, pass `jsonSchemaOutputFormat(schema, { transform: false })`.

23.4 Strict Tool Use: `strict: true`

`strict` is a top-level property of a tool definition, sitting alongside `name`, `description` and `input_schema`.

```
tools = [{
    "name": "get_weather",
    "description": "Get the current weather in a given location.",
    "strict": True,
    "input_schema": {
        "type": "object",
        "properties": {
```

```
            "location": {"type": "string"},
            "unit": {"type": "string",
                     "enum": ["celsius", "fahrenheit"]},
        },
        "required": ["location"],
        "additionalProperties": False,
    },
}]
```

The documentation's motivating example is the one everybody has hit: a booking tool that needs `passengers: int` and receives `"two"`. With `strict: true` the `input` field always matches the schema and the tool `name` is always one you provided or a server tool. Non-strict tools are unaffected and do not count towards the strict-tool limit, which makes a mixed request the normal case — mark the tools where a schema violation causes real damage, and let the rest rely on the model's ordinary adherence.

There is one combination worth knowing because it converts two probabilistic guarantees into two certainties. Setting `tool_choice: {"type": "any"}` forces *some* tool to be called; `strict: true` guarantees its arguments. Together you get a guaranteed, well-typed call — which is precisely the mechanism the documentation recommends for classification now that prefilling is gone. Define a tool with a single enum-typed field, force it, and read the label out of `input`. Chapter 16 listed that as one of prefill's five replacements and deferred the detail here.

> WARNING
>
> **Forced tool use and manual thinking do not mix**
>
> Chapter 18 recorded that manual extended thinking (`thinking: {"type": "enabled"}`) supports only `tool_choice` `auto` or `none`. That restriction is unchanged by `strict`. Adaptive thinking — the default on Fable 5, Mythos 5, Opus 5 and Sonnet 5 — does support forced tool use. Claude Mythos Preview does not support forced tool use at all and returns a 400.

23.5 The Supported Subset of JSON Schema

JSON Schema is a large specification and the grammar compiler accepts a defined slice of it. Everything below is transcribed from the documentation's own two lists, read on 1 August 2026.

Supported. All the basic types — `object`, `array`, `string`, `integer`, `number`, `boolean`, `null`. `enum`, restricted to strings, numbers, booleans or nulls. `const`. `anyOf` and `allOf`, with the caveat that `allOf` combined with `$ref` is not supported. `$ref`, `$defs` and `definitions`, internal only. `default` for every supported type. `required`, and `additionalProperties` — which **must be present and must be** `false` on every object. Ten string formats: `date-time`, `time`, `date`, `duration`, `email`, `hostname`, `uri`, `ipv4`, `ipv6`, `uuid`. And array `minItems`, for the values 0 and 1 only.

Not supported. Recursive schemas. Complex types inside enums. External `$ref` such as `http://…`. All numerical constraints — `minimum`, `maximum`, `multipleOf`. All string length constraints — `minLength`, `maxLength`. Any array constraint beyond `minItems` of 0 or 1. And `additionalProperties` set to anything other than `false`. Using one of these returns a 400 with details.

Patterns get their own list, because `pattern` is supported but the regex dialect is not the full one. Supported: full matching with `^…$` and partial matching, the quantifiers `*`, `+`, `?` and simple `{n,m}`, the

character classes `[]`, `.`, `\d`, `\w`, `\s`, and groups `(...)`. Not supported: backreferences such as `\1`, lookahead and lookbehind, the word boundaries `\b` and `\B`, and complex `{n,m}` with large ranges.

Now the interesting part, which is a keyword that appears in *neither* list.

> COMMON MISTAKE
>
> **`oneOf` is not documented either way**
>
> **Symptom.** A schema using `oneOf` sometimes compiles and sometimes 400s, and nobody can find the rule.
>
> **Cause.** The documentation lists `anyOf` and `allOf` as supported and does not mention `oneOf` in the unsupported list either. That is not permission; it is silence.
>
> **Fix.** Rewrite as `anyOf` if the branches are genuinely alternatives, or as an enum-tagged union if they are variants of one record. Do not ship the `oneOf`.
>
> **Detection.** A schema reader that classifies every keyword into supported, unsupported or **unknown**, and raises on unknown. Chapter 14 established the rule for model facts — *a fact is verified on a date or it is UNKNOWN, and UNKNOWN must raise rather than default* — and it applies to schema keywords without modification. Section 23.16 does exactly this; `oneOf` is one of the seventeen rejections it catches offline.

23.6 The Three Limits You Can Count, and the Ones You Cannot

Schema complexity is capped because grammar compilation is not free, and the documentation is unusually candid that the caps are protective rather than principled.

Limit	Value	Scope
Strict tools per request	**20**	tools with `strict: true`; non-strict tools do not count
Optional parameters	**24**	**combined** across all strict tool schemas *and* the JSON output schema
Union-typed parameters	**16**	anything using `anyOf` or a type array, same combined scope
Compilation timeout	**180 seconds**	a final stop-gap

The scope is the part that catches people. These are not per-schema limits. The documentation spells the trap out: four strict tools with six optional parameters each reach the twenty-four-parameter limit even though no individual tool looks complex. An optional parameter is expensive because the grammar must encode both the branch where it appears and the branch where it does not; the documentation's phrasing is that each one *"roughly doubles a portion of the grammar's state space"*. Union types are worse — *"especially expensive because they create exponential compilation cost"*.

Beyond the countable limits there are internal ones on compiled grammar size, and exceeding them returns a 400 saying **"Schema is too complex for compilation."** The reason given is honest and worth quoting, because it explains why no single number would have done: *"schema complexity doesn't reduce to a single dimension: features like optional parameters, union types, nested objects, and number of tools interact with each other."*

The documented remedies, in the order the documentation gives them: mark only critical tools as strict; make optional parameters required and have the model supply the default explicitly; flatten nested

structures; and split many strict tools across separate requests or subagents. That last one is a forward reference — orchestrator-and-subagent topologies are Chapter 50 — and it is the first time in this book that a *schema* has been a reason to change your architecture.

23.7 Four Things the Schema Does Not Promise

This is the section to reread before you ship.

Refusals. If Claude declines a request on safety grounds, `stop_reason` is `"refusal"`, the HTTP status is **200**, you are billed for the tokens generated, and *the output may not match your schema, because the refusal message takes precedence over the schema constraints*. Chapter 16 established that a refusal is a successful HTTP response with a control-flow meaning; structured outputs do not change that, they add a case where the body is prose in a channel your code believes is JSON.

Truncation. If the response hits `max_tokens`, `stop_reason` is `"max_tokens"` and the output is incomplete and therefore does not match your schema. The documented fix is to retry with a higher `max_tokens`. Note what is *not* on offer: there is no partial-object recovery, and Section 23.21 measures what happens when you build one anyway.

Enum capitalisation. Structured outputs *"don't guarantee the capitalization of string enum and const values"*. The documentation's own example is a three-member enum ending `"Conversation topic 3"` and an output of `"Conversation Topic 3"` — a capital T on a word following a space, a value that is not in the enum. And then the sentence that makes it dangerous: *"The response completes normally, with no error and no special stop_reason."* The guidance is to compare case-insensitively and to avoid enum values that differ only in capitalisation. Both halves matter: the first is how you read one, the second is a schema-design rule, because two members differing only in case are not distinguishable in the output.

Property ordering. Properties keep your declared order with one caveat: **required properties come first, then optional ones.** Declare `notes, name, email, age` with `name` and `email` required and the output arrives as `name, email, notes, age`. This is harmless for anyone using a JSON parser and fatal for anyone who has written `list(obj.values())[0]`, diffed two serialisations, or hashed the response body for idempotency. The documented fix is to mark everything required, or to account for the reordering.

> KEY TAKEAWAYS
>
> **The four exceptions, compressed**
>
> Two of them are `stop_reason` values, which means the branch you already wrote in Chapter 16 handles them. Two of them are *silent*: enum casing and property ordering produce a normal 200 with no signal at all. The silent ones are the expensive ones.

23.8 Incompatibilities, and What Still Caches

Citations return a 400. Chapter 8 introduced the citations feature and what it guarantees — pointers into your documents that are valid by construction. It cannot be combined with `output_config.format`: citations work by interleaving citation blocks with text, and a strict JSON grammar has nowhere to put them. This is the single most consequential incompatibility in the chapter, because it means a

cited-answer pipeline cannot use JSON outputs at all and stays entirely in Chapter 22's world of envelopes and structural parsing.

Prefilling is incompatible with JSON outputs — and is separately gone on Claude 4.6 and later, as Chapter 16 recorded. The old trick of prefilling `{` to force JSON is dead twice over, and structured outputs are its documented replacement.

What works. Batch processing, at the usual 50 per cent discount. Token counting, which the documentation notes runs *without compilation*. Streaming, which behaves exactly as Chapter 17 described — the JSON arrives as `text_delta` events and is only a document once `message_stop` has fired. And the two structured-output features together in one request.

Two cost facts round it out, and neither is optional reading. First: **using structured outputs injects an additional system prompt** describing the expected format, which costs input tokens like any other system prompt. The documentation publishes no size for it, and Section 23.22 reports a sensitivity rather than inventing a number. Second: **changing `output_config.format` invalidates the prompt cache for that thread.** Chapter 20's rule — stable parts first, variable parts last — now has a new member, and it is one of the most stable parts you have.

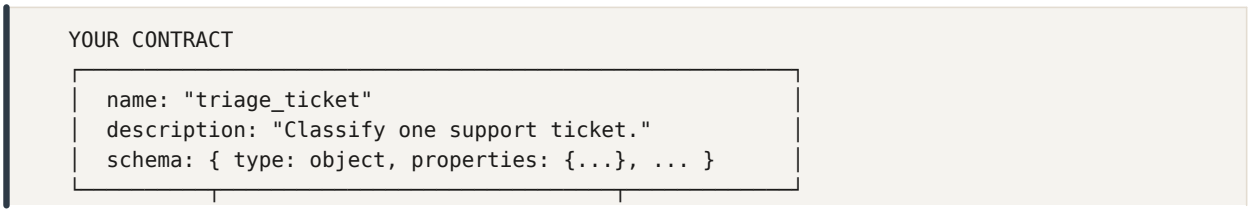
23.9 Two Caches, Two Keys

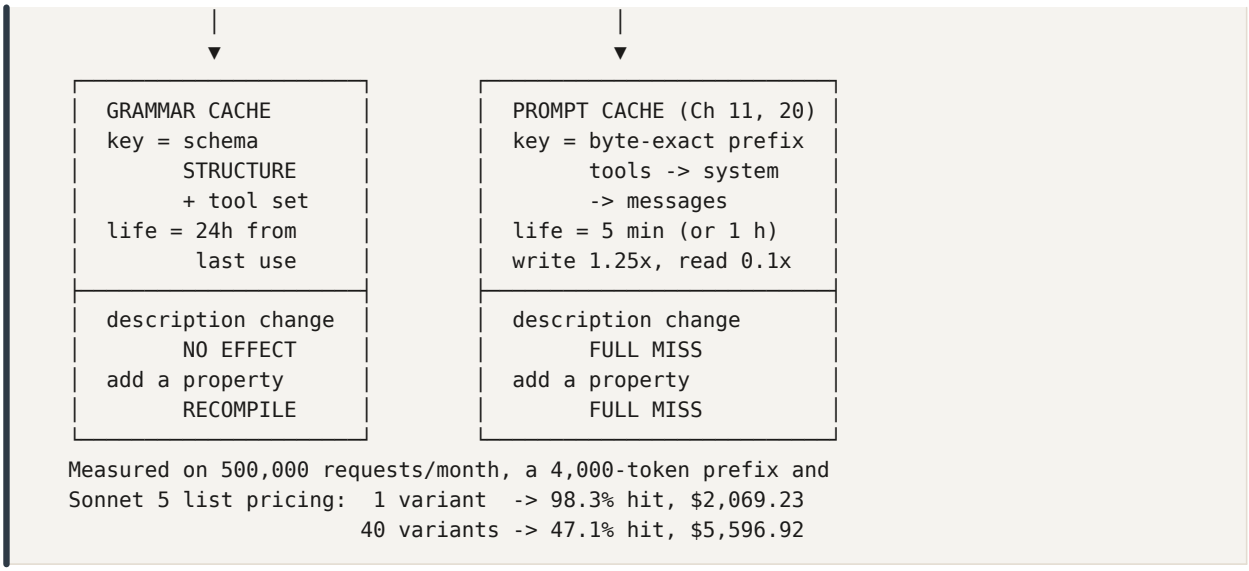
Chapter 11 introduced the prompt cache and Chapter 20 measured what happens when you disturb its prefix. Structured outputs add a **second** cache, and the two have different keys. Confusing them is expensive in a way that is invisible in code review.

> DEFINITION
>
> ### The grammar cache
>
> Compiled grammars are cached for **24 hours from last use**. The first request with a given schema pays a compilation latency; subsequent ones do not. The cache is invalidated by a change to the **JSON schema structure** or to the **set of tools** in the request. Changing only `name` or `description` fields **does not** invalidate it.

Read that last sentence next to Chapter 20's rule about the prompt cache and the collision is obvious. The prompt cache is keyed on the byte-exact prefix; the schema is *in* that prefix. So a per-tenant `description` string — "billing category for the Acme team" — is completely invisible to the grammar cache and completely fatal to the prompt cache.

Section 23.22 prices it on a 500,000-request month. Forty tenant variants that differ **only** in description strings compile to exactly **one** grammar, and take the prompt-cache hit rate from **98.3 per cent to 47.1 per cent**, taking the monthly bill from **$2,069.23 to $5,596.92** — an increase of **170.5 per cent** for an edit that one of the two caches never noticed.





23.10 Where the Feature Is Available

Structured outputs are generally available on the Claude API for **Claude 4.5 and later** and for Claude Mythos Preview. Beyond that the availability is per platform and per model, and it is not the same list twice.

Platform	Documented position, 1 August 2026
Claude API	GA for Claude 4.5 and later, plus Mythos Preview
Amazon Bedrock	GA for Opus 5, Opus 4.8, Opus 4.6, Sonnet 4.6, Sonnet 4.5, Opus 4.5, Haiku 4.5; Sonnet 5, Opus 4.7 and Mythos Preview through *Claude in Amazon Bedrock*, the Messages-API endpoint
Claude Platform on AWS	available; **no model list is published**
Google Cloud	GA for Fable 5, Mythos 5, Opus 5, Opus 4.8, Mythos Preview, Opus 4.7, Opus 4.6, Sonnet 5, Sonnet 4.6, Sonnet 4.5, Opus 4.5, Haiku 4.5
Microsoft Foundry	GA; **requires a Hosted on Anthropic deployment**

Two things fall straight out of that table and both are running threads. **Fable 5 and Mythos 5 appear on the Google Cloud list and on neither Bedrock list.** That is not a statement that they are unsupported there; it is silence, and Chapter 14's rule says silence is UNKNOWN. And *Claude Platform on AWS* is documented at feature level with no per-model detail at all.

Section 23.17 builds the matrix — twelve models by five platforms, sixty cells — and finds **14 of 60 UNKNOWN**: two on Bedrock and all twelve on Claude Platform on AWS. Every one of them raises rather than defaulting to `True`. **Capability does not travel in a bundle**, and it does not travel across platforms either.

23.11 Migrating Chapter 22's Envelope

Chapter 22 ended with an output envelope: `<answer>` for the answer, `<quotes>` for the supporting spans, and a structural parser that got 9 of 9 where a greedy regex got 7 and a lazy one got 6. The migration to a schema deletes that parser. It also deletes something else, and it is worth naming before you press the button.

What you gain. Parsing disappears entirely — not "improves", disappears. The `<answer>` element with an attribute on it and the response that quotes a document containing `<answer>`, the two failures Chapter 22 measured, cannot occur, because the response is a JSON document produced by a grammar. Delimiter collision from untrusted document content, the first half of Chapter 22, is also gone from the *output* side: nothing the model emits can close a container, because there are no containers.

What you lose. The envelope was free-form. Prose could sit alongside the answer, a model could add a caveat in a place you had not anticipated, and you could read it. A schema has no such place. Anything not in `properties` cannot be emitted, because `additionalProperties` must be `false`.

How to keep it. Put the free-form channel *inside* the schema. Add `"quotes": {"type": "array", "items": {"type": "string"}}` and the supporting spans come back as data instead of as markup. Add `"notes": {"type": "string"}` and the model has somewhere to put a caveat. The difference from Chapter 22 is that you must decide in advance that the channel exists — which is a real loss of flexibility, and the right trade in almost every production system.

What still needs Chapter 22. Three things, precisely. The **refusal** path, where the body is prose. The **max_tokens** path, where the body is a fragment. And any pipeline using **citations**, which cannot use JSON outputs at all and therefore stays on the envelope end to end. Section 23.21 measures the first two honestly and reports a result that surprised the author: Chapter 22's structural parser recovers **0 of 2**. On those paths a better parser is not the answer; control flow is.

23.12 Visual Analogies

> VISUAL ANALOGY
>
> ### The corridor and the bouncer
>
> Post-hoc validation is a bouncer on the door of a club. People arrive dressed however they like, and the bouncer turns some away. You pay for everyone's journey, including the ones who get turned away, and the queue is unpredictable.
>
> Constrained decoding is a corridor with no wrong doors. There is no bouncer because there is nothing to turn away: every route through the building ends somewhere legal.
>
> **Where it breaks.** A corridor decides where you can walk, not where you *should*. Constrain a triage output to `S1`–`S4` and you get one of those four every time, including on the ticket that should have been `S1` and came back `S3`. The building is guaranteed; the destination is not. And the corridor has two fire exits — a refusal and a token limit — that open onto the street.

VISUAL ANALOGY

The form and the printing press

Chapter 22's envelope was a form you asked someone to fill in. Most people fill it in properly. Some write outside the boxes, some fold it, one person writes the word "box" inside a box and confuses your scanner.

A schema is not a better form. It is a printing press that can only print your form: the plate physically cannot produce a page with an extra field.

Where it breaks. The press guarantees the layout of the page and has no view on the content. It will print a beautifully laid-out page saying the customer's severity is S3 when it was S1, and it will print it faster and more cheaply than the handwritten version. It also has two states in which it prints nothing usable at all — jammed halfway, and refusing to run — and in both of those a page still comes out of the machine.

VISUAL ANALOGY

Two filing cabinets with different labels

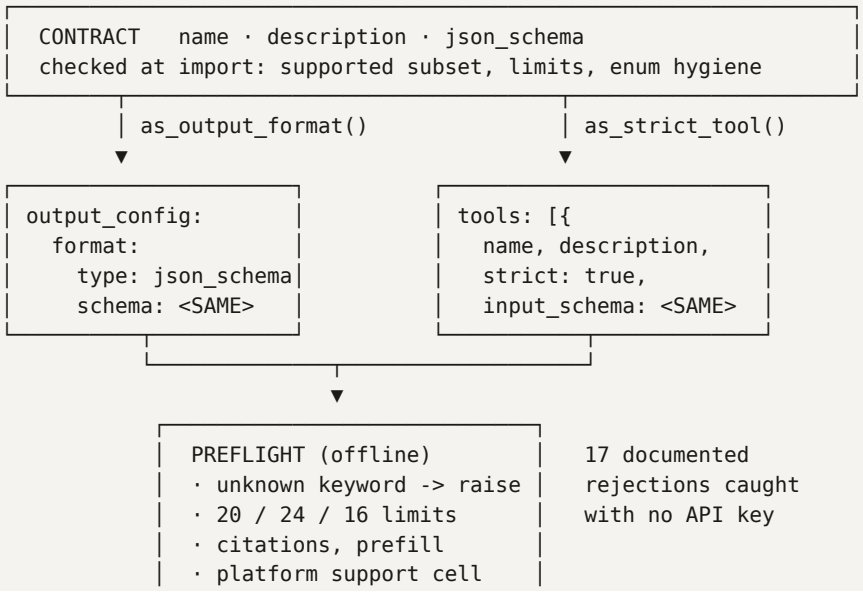
The prompt cache files things by the exact wording of the whole request prefix. The grammar cache files things by the *shape* of your schema. Retitle a folder and the first cabinet cannot find anything and the second does not care.

Where it breaks. Real filing cabinets do not charge you for a miss. These do: the measured cost of retitling forty folders is $3,527.69 a month, and the cabinet that did not care is the one people watch.

23.13 Architecture: One Contract, Two Emitters, One Reader

The failure mode this architecture exists to prevent is duplication. A team writes a JSON Schema for the `output_config.format`, writes a second one for the strict tool that does the same job, writes a Pydantic model for the application layer, and writes a validator for the response. Four descriptions of one contract, drifting apart at four different rates.

The fix is a single object — a **contract** — that owns the schema, emits both request shapes, and is the only thing the reader validates against.



```
                └──────────────┬──────────────┘
                               ▼  HTTP
                ┌──────────────────────────────┐
                │  GUARDED READER              │
                │  1. branch on stop_reason    │──► refusal    ──► policy path
                │  2. json.loads               │──► max_tokens──► retry higher
                │  3. validate (lenient enum)  │
                │  4. canonicalise enums       │
                │  5. hand to the application  │
                └──────────────────────────────┘
```

Step 1 of the reader is the one teams skip, and it is the one that matters. **Branch on `stop_reason` before you parse.** Chapter 16 said it about `content[0].text`; it is even more true when a schema has convinced everyone that parsing is now safe.

23.14 Examples

A classifier without prefill. One enum field, one forced tool, and the label arrives typed.

```
tool = {"name": "label", "description": "Record the label.",
        "strict": True,
        "input_schema": {
            "type": "object",
            "properties": {"label": {"type": "string",
                           "enum": ["spam", "ham", "unsure"]}},
            "required": ["label"], "additionalProperties": False}}
body = {"model": "claude-haiku-4-5", "max_tokens": 64,
        "messages": [{"role": "user", "content": subject}],
        "tools": [tool], "tool_choice": {"type": "any"}}
```

Reading an enum, correctly. Never `==`.

```
def enum_match(value, allowed):
    for a in allowed:
        if isinstance(a, str) and a.lower() == value.lower():
            return a
    return None
```

Counting optional parameters before the API does. Twelve tools with two optional fields each is twenty-four — the limit, exactly, with nothing left for the output schema.

```
optional = sum(len(t["input_schema"]["properties"])
               - len(t["input_schema"]["required"])
               for t in strict_tools)
assert optional <= 24, f"{optional} optional parameters"
```

23.15 Code Walkthrough: The Schema Reader That Refuses to Guess

The heart of the project is a walker over a JSON Schema that classifies every keyword it meets. It is thirty lines of the file and it is the thirty lines that pay for themselves.

```
for key in node:
    if key in ANNOTATIONS or key in SUPPORTED_KEYWORDS:
        continue
    if key in UNSUPPORTED_KEYWORDS:
        fix = ("stripped by the SDK and moved into the description"
               if key in SDK_STRIPPED else "no SDK fallback")
        rep.add("error", "UNSUPPORTED", f"{path}.{key}",
                f"{UNSUPPORTED_KEYWORDS[key]}; {fix}")
    else:
        rep.add("error", "UNKNOWN_KEYWORD", f"{path}.{key}",
                "not in the documented supported or unsupported "
                "lists; refusing to guess")
```

Line by line. The loop is over keys, not over a schema object model, because the point is to notice keys nobody anticipated. `ANNOTATIONS` — `title`, `description`, `$comment` and friends — are waved through: they constrain nothing, and they are also exactly the fields the grammar cache ignores, which matters in Section 23.22. `SUPPORTED_KEYWORDS` is the documented list, transcribed. `UNSUPPORTED_KEYWORDS` maps each documented exclusion to the reason the documentation gives, and the message distinguishes the five that a Python or TypeScript SDK will silently strip into the description from the ones with no fallback at all — because "this will 400" and "this will quietly stop being enforced by the grammar" are different problems for the reader of the error.

The `else` branch is the one that matters. Anything in neither list is `UNKNOWN_KEYWORD`, and `Contract.check()` raises `UnknownSchemaFeature` on it rather than continuing. `oneOf` lands here. So does any keyword a future specification adds, and so does a typo — `requred` is not a supported keyword, and a schema reader that silently ignores unknown keys will happily send a schema with no required fields at all.

The counting is equally deliberate:

```
if not is_req:
    rep.optional_params += 1
    seen_optional = seen_optional or name
elif seen_optional:
    rep.add("warn", "ORDER_SHIFT", p,
            f"required, but declared after optional "
            f"{seen_optional!r}; the output puts required "
            f"properties first")
if _is_union(sub):
    rep.union_params += 1
```

`optional_params` and `union_params` are accumulated on the report, not returned per node, because the API's limits are combined across every strict schema and the JSON output schema in one request. A per-schema counter would pass twelve tools with two optional fields each and then take a 400.

`ORDER_SHIFT` is not an error — the request is legal — but the moment an optional property is declared before a required one, the response key order stops matching the declaration order. That is the documented required-first behaviour, and it is a warning rather than an error because the correct response is usually "yes, I know, I use a JSON parser".

23.16 Mini Project: The Schema Compiler and Contract Harness

What it does. Takes a JSON Schema, reports what is inside the supported subset and what is not, emits both request shapes from one source of truth, enforces every documented limit offline, and then measures two things that are usually reported as one: whether responses conform, and whether they are right.

What it must not do. Make a network call. Everything below runs with no API key.

```
ch23/
├── schema.py        the supported subset; classify, count, raise
├── contract.py      one contract -> two shapes; limits; platforms
├── conform.py       a validator over the SAME subset; enum matching;
│                    the guarded reader; the truncation "repair"
├── fixtures.py      6 DOC + 6 BUILT fixture groups, provenance table
├── harness.py       conformance vs correctness; the exception paths
├── cost.py          the two caches, priced
├── report.py        prints every figure in this chapter
├── test_contract.py 39 tests
├── wire.py          copied from ch16/ — build_request, parse_response
├── envelope.py      copied from ch22/ — top_level(), the XML parser
└── planner.py       copied from ch19/ — rate-limit arithmetic
```

```
cd ch23 && python3 -m pytest -q      # 39 passed
python3 report.py                    # every published figure
```

The design rule the whole thing turns on: `conform.py` **validates only what the grammar compiler enforces.** A validator that also checked `minimum` and `maxLength` would report violations the platform never promised to prevent, and would make structured outputs look less reliable than they are. Measuring a guarantee against a stricter standard than the guarantee is a way of learning nothing.

23.17 Panel A — The Contract, Read Before It Is Sent

The contract under test is support-ticket triage. `notes` is declared **first** and is the only optional property, which makes the documented reordering visible.

```
properties 6  optional 1  unions 0  depth 2
budget: 1/20 strict tools, 2/24 optional, 0/16 unions
```

Level	Code	Where	What it means
WARN × 5	`ORDER_SHIFT`	the five required properties	each is declared after the optional `notes`, so the output will put it earlier than declared
INFO × 2	`ENUM_CASE`	`$.category`, `$.severity`	capitalisation is not guaranteed; compare case-insensitively

Note the budget line: **2 of 24 optional parameters** for one contract used twice, because the same schema is sent as a strict tool *and* as the output format and the limit is combined. This is the arithmetic that makes twelve two-optional-field tools a 400.

23.18 Panel B — Seventeen Rejections, No API Key

Every one of these is a request or a schema the documentation says will be rejected, or a fact the documentation does not state. All seventeen are caught before any bytes leave the process.

```
   CAUGHT  citations with output_config.format       ContractError
   CAUGHT  assistant prefill with JSON outputs       ContractError
   CAUGHT  21 strict tools                           ContractError
   CAUGHT  25 optional parameters                    ContractError
   CAUGHT  17 union-typed parameters                 ContractError
   CAUGHT  minimum on a number                       ContractError
   CAUGHT  maxLength on a string                     ContractError
   CAUGHT  external $ref                             ContractError
   CAUGHT  recursive schema                          ContractError
   CAUGHT  additionalProperties omitted              ContractError
   CAUGHT  additionalProperties: true                ContractError
   CAUGHT  an unsupported string format              ContractError
   CAUGHT  minItems: 3                               ContractError
   CAUGHT  an object inside an enum                  ContractError
   CAUGHT  a lookahead in a pattern                  ContractError
   CAUGHT  oneOf -- documented neither way           UnknownSchemaFeature
   CAUGHT  structured outputs on an UNKNOWN cell     UnknownSchemaFeature
17 of 17 caught with no API key
```

The last two raise a different exception on purpose. The first fifteen are things the documentation says are wrong. The last two are things the documentation does not say, and conflating "documented as invalid" with "not documented" is how a registry rots.

The support matrix is the same discipline applied to availability:

```
60 cells, 14 UNKNOWN
   api      UNKNOWN on  0 of 12 models
   bedrock  UNKNOWN on  2 of 12 models
   aws      UNKNOWN on 12 of 12 models
   google   UNKNOWN on  0 of 12 models
   foundry  UNKNOWN on  0 of 12 models
```

Twelve models, five platforms. The two Bedrock unknowns are **Fable 5 and Mythos 5**, which appear on the Google Cloud list and on neither Bedrock list. The twelve on Claude Platform on AWS are all of them, because the documentation states that the feature is available there and publishes no model list. `supported_on()` raises on every one of the fourteen. Defaulting them to `True` would produce a preflight that passes and a production request that 400s; defaulting them to `False` would produce a system that refuses work it could do.

23.19 Panel C — Conformance Is Not Correctness

Twenty-four BUILT tickets, a gold key, and two families of response: **governed** (as constrained decoding produces them) and **ungoverned** (the same twenty-four answers from a prompt-and-parse pipeline, carrying the four defect kinds the documentation names — parse errors, missing required fields, inconsistent types, schema violations).

```
family / schema              parse  conform  correct
governed / strict            24/24    24/24    15/24
```

```
governed / strict, enum ==    24/24    21/24    12/24
ungoverned / loose            22/24    20/24    12/24
ungoverned / strict           22/24    14/24     9/24
judgement ceiling: 15/24 = 62.5% -- identical for both families
undeclared fields admitted by the loose schema: 6
```

Read the table in three passes.

Pass one, the win. Governed responses parse 24 of 24 and conform 24 of 24. Ungoverned responses parse 22 — one wrapped in a Markdown fence, one with a trailing comma — and conform 20 once a string `"0.9"` and a missing `customer_email` are counted. End to end, **12 of 24 correct becomes 15 of 24**. That is a 12.5-point gain in delivered correctness for twenty lines of schema, and it is why you should do this.

Pass two, the ceiling. The judgement ceiling — how many answers are right, ignoring format entirely, with enum casing normalised away — is **15 of 24 in both families**. The grammar moved delivered correctness from 50.0 per cent to 62.5 per cent by rescuing three answers that were right all along and were lost in transit. It moved the ceiling by exactly zero. The nine wrong answers were wrong before the migration and are wrong after it, and now they are wrong in beautifully typed JSON.

Pass three, the trap. The second row is the same governed responses read by a team that compares enum values with `==`. Conformance drops to **21 of 24** and correctness to **12 of 24** — the migration's entire benefit, given back, because the three drifted values were all on tickets the model got right.

> KEY TAKEAWAYS
>
> **The three numbers**
>
> Parse rate is a property of your transport. Conformance is a property of the platform. Correctness is a property of the model and your prompt. Structured outputs move the first two to 100 per cent and the third not at all, and a dashboard that shows only the first two will show a perfect score during an outage of judgement.

The final line is the quiet one. The ungoverned family carried an undeclared `priority_score` on six responses. Validated against the loose schema — the one without `additionalProperties: false`, which is what most post-hoc validators are handed — all six pass, and the application reads none of them: **six fields, generated, billed, ignored.** Validated against the real schema, conformance falls to 14 of 24. Structured outputs close this hole by construction, because `additionalProperties` must be `false` and the grammar therefore cannot emit the field at all.

23.20 Panel D — The Enum That Was Never in the Enum

```
enum drift on governed responses:
   row  3: 'feature Request' -> 'feature request'
   row 11: 'Feature request' -> 'feature request'
   row 16: 'account Access' -> 'account access'
```

Three of twenty-four, and each is exactly the documented pattern: a capital letter on a word following a space. There is no error, no `stop_reason` to branch on, and the response completes normally. The documentation's own example is transcribed as a DOC fixture and asserted:

```
DOC enum example 'Conversation Topic 3' in enum: False
                 case-insensitive match: 'Conversation topic 3'
```

Two rules follow, and they are different rules. **When reading**, match case-insensitively and then canonicalise to the declared spelling before the value goes anywhere near a switch statement, a database column or a metric label. **When designing**, never put two members in one enum that differ only in capitalisation — `["Draft", "draft"]` cannot be read back reliably by anyone. The schema reader treats the first as an INFO on every enum and the second as an **error**.

The DOC property-ordering fixture is asserted in the same panel:

```
DOC declaration order ['notes', 'name', 'email', 'age']
DOC output order      ['name', 'email', 'notes', 'age']
```

23.21 Panel E — The Two Paths With No Grammar

```
   refusal      naive=JSONDecodeError  guarded=refusal     xml elements=0
   max_tokens   naive=JSONDecodeError  guarded=max_tokens  xml elements=0
```

The naive reader — `json.loads(response.all_text())`, written by a team that has been told the platform guarantees the schema — raises on **both**. That is the loud failure, and Chapter 17's rule holds: the loud failure is the cheap one. The guarded reader branches on `stop_reason` first and never reaches the parser.

Now the negative result, which is not what the author expected to publish.

Chapter 22's structural parser — the one that beat both regexes 9 to 7 and 9 to 6 — recovers **0 of 2** here. It finds no elements in the refusal, because a refusal is a plain sentence of prose with no tags in it, and none in the truncated body, because a truncated JSON object has no tags either. On these two paths a better parser is not the answer. **Control flow is.** The place Chapter 22's parser genuinely survives is elsewhere: a pipeline that needs citations cannot use `output_config.format` at all, and stays on the envelope end to end.

Then the tempting fix, measured:

```
   repair of the truncated body: parses=True, conforms=False
      violation: $.customer_email: required, missing
   with a default filled in: conforms=True,
      email='unknown@example.com', quote ends 'n down since 0'
```

Closing the open string, the open array and the open object turns a truncated fragment into a document that **parses**. It does not conform, because a required field is missing. So somebody adds a default — and now it conforms, and it carries a fabricated email address and a quotation that ends mid-word in the middle of a sentence. The repair made it parse, and parsing was never the problem.

COMMON MISTAKE

Repairing a truncated structured response

Symptom. No parse errors in the logs, and a slow drip of records with placeholder values and half-sentences.

Cause. `stop_reason: "max_tokens"` was treated as a formatting problem. Bracket-closing plus defaults produces a schema-conformant object that is fiction.

Fix. Retry with a higher `max_tokens`. That is the documented remedy and there is no other one.

Detection. Assert that every accepted record came from a response with a terminal `stop_reason`. Count `max_tokens` as its own metric; if it is non-zero your `max_tokens` is too low for the schema you are asking for, and a large schema with many required fields needs more room than the prose it replaced.

23.22 Panel F — Two Caches, Two Keys, Priced

Forty tenants, each wanting the field descriptions in their own vocabulary. The schema *structure* is identical for all forty.

```
40 description-only variants compile to 1 grammar
    1 prefix variant(s):  57.8704 arrivals/window, hit 98.3%, $2,069.23
   40 prefix variant(s):   1.4468 arrivals/window, hit 47.1%, $5,596.92
   delta $3,527.69 (+170.5%) for an edit the grammar cache never noticed
```

The model is deterministic and closed-form, not simulated: with `lam` requests expected inside one five-minute cache lifetime, a window with $k \geq 1$ arrivals pays one write and $k - 1$ reads, so the expected share of writes is $(1 - e^{-})/$. At 500,000 requests a month the whole service sees 57.87 arrivals per window and almost everything is a read. Split forty ways, each tenant sees 1.45, and **more than half of all requests pay a cache write at 1.25× instead of a read at 0.1×** — a twelve-and-a-half-fold difference on 4,000 prefix tokens, on every one of them.

The grammar cache, meanwhile, is entirely content: forty variants, **one grammar**, because it is keyed on structure and the documentation states plainly that changing only `name` or `description` does not invalidate it. Every engineer on that team could have been right about the cache they were thinking of.

The calendar is in the number, as it was in Chapter 14:

```
   at introductory pricing (to 31 Aug 2026): $1,379.49
   from 1 September:                          $2,069.23
```

And the honest gap. The documentation states that structured outputs inject an additional system prompt describing the expected format, and that it costs input tokens like any other system prompt. **It publishes no size.** Rather than invent one, the harness reports a sensitivity:

```
   injected system prompt, unpublished size:
       100 tokens -> +$17.98/month
       200 tokens -> +$35.96/month
       400 tokens -> +$71.92/month
```

At this workload the injection is a rounding error next to the caching decision — between 0.9 and 3.5 per cent of the bill — which is itself the useful finding. Spend your attention on the prefix, not on the injection.

23.23 Evaluation and Deployment

Evaluate three numbers, not one. Parse rate, conformance rate and correctness rate, from the same responses, reported separately. If your evaluation harness collapses them, the migration in Section 23.19 looks like a 100 per cent success and a 37.5 per cent error rate goes unmeasured.

Deploy the preflight in CI, not at runtime. `Contract.check()` is a pure function of the schema. Run it as a test; a schema that raises at import in production is a schema you shipped without reading.

Roll out on one route. Structured outputs change the injected system prompt, the cache prefix and the compiled grammar all at once. Move one route, compare correctness against the same evaluation set, and only then move the rest.

Watch two new metrics. `stop_reason == "max_tokens"` as a rate, because a schema with many required fields needs more output room than the prose it replaced; and enum-drift count, because it is invisible unless you count it.

23.24 Hands-on Labs

Lab 1 — Read a schema before you send it. Take a JSON Schema from a project you already have and run it through `schema.analyse`. **Success criterion:** you can name every keyword in it that is unsupported, and every one that is documented neither way.

Lab 2 — One contract, two shapes. Build a `Contract` and assert that `as_output_format()["format"]["schema"]` and `as_strict_tool()["input_schema"]` are the *same object*, then add a property and confirm both change. **Success criterion:** one edit, two shapes, no duplication.

Lab 3 — Break each limit deliberately. Construct requests with 21 strict tools, 25 optional parameters and 17 unions, and confirm the preflight raises with a message naming the documented limit. **Success criterion:** three distinct errors, no network calls.

Lab 4 — Measure the enum trap. Run the twenty-four governed fixtures with `strict_enum=True` and again with `False`. **Success criterion:** you can state, in points of correctness, what `==` costs — and explain why it is not zero.

Lab 5 — Price your own prefix split. Put your real request volume, prefix size and variant count into `cost.plan`. **Success criterion:** a monthly figure for consolidating variants, and a statement of which cache each change affects.

23.25 Lab Solutions

1. `S.analyse(sch)` then filter `rep.findings` by code. `UNSUPPORTED` findings tell you what will 400 and which of them a Python or TypeScript SDK would have stripped into the description; `UNKNOWN_KEYWORD`

findings are the ones to research rather than guess. Real schemas almost always contain `minLength` or `maximum` somewhere, and almost always contain at least one object missing `additionalProperties: false`.

2. `Contract` stores the dict and both emitters hand out a reference, so identity holds — `test_both_shapes_carry_the_same_schema_object` asserts it. If you find yourself wanting a deep copy, that is the moment to notice you are about to have two contracts.

3. `compile_request` raises `ContractError` for all three. Note that the optional and union counts are summed across strict tools *and* the output schema before comparison, which is what makes `budget()` worth calling in CI.

4. Lenient reading delivers 15 of 24 correct; strict `==` delivers 12. The three lost answers are rows 3, 11 and 16, all of which were right. The cost of `==` is **12.5 points of correctness**, and it is not zero because casing drift is uncorrelated with whether the answer was any good.

5. `cost.plan(variants=n)`. The lever is variant count, not prefix size: halving the prefix helps linearly, while consolidating variants moves the hit rate, which is multiplicative.

23.26 Best Practices

BEST PRACTICE

One schema object, two emitters

Never write the JSON output schema and the strict tool schema separately. They drift, and the drift shows up as a 400 in whichever one you edit less often.

BEST PRACTICE

Compare enum values case-insensitively, always

The documentation guarantees the value is *in* your enum; it does not guarantee the capitalisation. Canonicalise on read, before the value touches a switch, a database column or a metric label.

BEST PRACTICE

Never put two enum members that differ only in case

`["Draft", "draft"]` cannot be read back reliably. This is a schema-design rule, not a reading rule, and no amount of careful parsing fixes it.

BEST PRACTICE

Mark everything required, and use `default` rather than optionality

Each optional parameter roughly doubles part of the grammar's state space, counts against the shared limit of 24, and triggers the required-first reordering. A required field with a documented default costs none of that.

BEST PRACTICE

Branch on `stop_reason` before you parse

Two of the seven values mean no grammar was enforced. This is Chapter 16's rule and structured outputs make it *more* important, not less, because they make everyone confident that parsing is now safe.

BEST PRACTICE

Reserve `strict: true` for the tools where a violation does damage

Non-strict tools do not count against the limit of twenty and do not add to grammar size. Strictness is a budget, not a default.

BEST PRACTICE

Keep the schema byte-identical across tenants

Descriptions feel free because the grammar cache ignores them. The prompt cache does not. Put tenant vocabulary in the system prompt or the user turn, never in the schema.

BEST PRACTICE

Preflight in CI, and fail the build on UNKNOWN

A keyword documented neither way, or a platform cell with no published position, should stop a deployment rather than default to a guess.

BEST PRACTICE

Give a schema more `max_tokens` than the prose it replaced

JSON with six required fields and an array of quotations is more tokens than a paragraph. Truncation is the exception path with no recovery.

BEST PRACTICE

Measure correctness on a held-out set that the schema cannot influence

Conformance will read 100 per cent from day one. If that is the only number you have, you have stopped measuring the model.

BEST PRACTICE

Count undeclared fields during migration

Anything your post-hoc validator was letting through is about to stop arriving, because `additionalProperties` must be `false`. Six of twenty-four, in the measurement above.

BEST PRACTICE

Do not put anything sensitive in a schema

Property names, enum values, `const` values and `pattern` regexes are cached separately from message content and, the documentation says explicitly, do not receive the same PHI protections.

BEST PRACTICE

Use token counting to price a schema

`count_tokens` works with structured outputs and, the documentation notes, runs without compilation — so you can price the injected prefix without paying the compile.

BEST PRACTICE

When citations are required, stay on Chapter 22

The combination returns a 400. That is a design constraint, not a bug to work around, and the envelope-and-parser design is the supported answer.

23.27 Common Mistakes

COMMON MISTAKE

Believing the schema is a promise about the answer

Symptom. Conformance dashboards at 100 per cent, and a steady stream of wrong decisions downstream.

Cause. Constrained decoding restricts the token distribution. It has no access to truth.

Fix. Keep a correctness metric measured against a gold key, separately from conformance.

Detection. If your evaluation harness reports one number for "schema" and none for "right", you have this bug now.

COMMON MISTAKE

Comparing enum values with `==`

Symptom. A small, stable rate of "invalid category" errors on responses that completed normally with no error and no special `stop_reason`.

Cause. Capitalisation of enum and `const` values is explicitly not guaranteed.

Fix. Case-insensitive match, then canonicalise to the declared spelling.

Detection. Log the raw value alongside the canonical one and count the disagreements. Three in twenty-four, in Section 23.20.

COMMON MISTAKE

Parsing before branching on `stop_reason`

Symptom. `JSONDecodeError` in production on a feature that guarantees valid JSON.

Cause. `refusal` and `max_tokens` are documented exceptions where the output may not match the schema, and both are HTTP 200.

Fix. Branch first. Refusals go to the policy path; truncation retries with more room.

Detection. Assert in tests that a refusal fixture never reaches the parser.

COMMON MISTAKE

Counting optional parameters per schema

Symptom. A 400 on a request where every individual tool looks simple.

Cause. The limit of 24 is combined across all strict schemas *and* the JSON output schema in one request.

Fix. Sum before you send; `budget()` does it in one call.

Detection. A CI assertion on the aggregate, run against the full tool set rather than tool by tool.

COMMON MISTAKE

Assuming `oneOf` is supported because it was not forbidden

Symptom. Intermittent 400s that correlate with which branch of the codebase built the schema.

Cause. The documentation lists `anyOf` and `allOf`; `oneOf` appears in neither list.

Fix. Rewrite as `anyOf`, or as an enum-tagged union.

Detection. Raise on unknown keywords. Silence is not permission.

COMMON MISTAKE

Per-tenant descriptions inside the schema

Symptom. Costs rise sharply after a "cosmetic" change, with no change in traffic, latency or grammar-compilation metrics.

Cause. The grammar cache ignores `description`; the prompt cache is keyed on the byte-exact prefix and does not.

Fix. One schema for all tenants. Tenant vocabulary belongs in the system prompt or the user turn.

Detection. Track `cache_creation_input_tokens` against `cache_read_input_tokens`. In Section 23.22 the write share goes from 1.7 per cent to 52.9 per cent.

COMMON MISTAKE

Letting the SDK rewrite the schema and then reasoning about the wire

Symptom. A cached prefix that never hits, or a `minimum` that is not enforced.

Cause. The Python, TypeScript, Ruby and PHP SDKs strip unsupported constraints into descriptions and add `additionalProperties: false` before sending.

Fix. Know which mode you are in. Build the schema yourself, or pass `{ transform: false }`, when the bytes matter.

Detection. Log the serialised body once at startup and compare it with the schema you wrote.

COMMON MISTAKE

Enabling citations and structured outputs together

Symptom. A 400 as soon as grounding is switched on.

Cause. Citations interleave citation blocks with text; a strict JSON grammar cannot express that.

Fix. Choose. Cited answers keep Chapter 22's envelope; schema-constrained answers give up citation pointers.

Detection. A preflight assertion, which is one line and catches it at build time.

COMMON MISTAKE

Treating a truncated response as a formatting problem

Symptom. Records with placeholder values and quotations that end mid-word.

Cause. Bracket-closing repair plus defaults produces a conformant object that is fiction.

Fix. Retry with higher `max_tokens`.

Detection. Every accepted record must trace to a terminal `stop_reason`.

COMMON MISTAKE

Assuming platform parity

Symptom. A schema that works on the Claude API and 400s on Amazon Bedrock.

Cause. Model availability for structured outputs differs by platform, and two current models appear on the Google Cloud list and on neither Bedrock list.

Fix. A per-(model, platform) matrix with UNKNOWN cells that raise. Fourteen of sixty, in Section 23.18.

Detection. Run the preflight with the platform as an argument, in the deployment's own CI.

23.28 Performance Considerations

Compilation is a first-request cost, not a per-request one. The documentation states that the first request with a given schema pays extra latency while the grammar compiles, and that compiled grammars are cached for **24 hours from last use**. A schema used once an hour never recompiles; a schema used once a day is on the boundary, and a schema used weekly pays every time. If a rare-but-latency-sensitive route uses its own schema, keep it warm.

Invalidation is not symmetrical with the prompt cache. The grammar cache is invalidated by schema *structure* and by the set of tools; `name` and `description` changes do not invalidate it. The prompt cache is invalidated by any byte change anywhere in the prefix. Section 23.22 prices the gap at 170.5 per cent.

Complexity is superadditive. Optional parameters, unions, nesting depth and tool count interact, which is why the documentation ships a 180-second compilation timeout and a "Schema is too complex for compilation" 400 in addition to the three countable limits. If you are near any of them, reduce optionality first: it is the cheapest change and it also removes the required-first reordering.

Constrained decoding does not make generation cheaper. Chapter 11's model is unchanged — decode dominates wall-clock time, and the grammar removes candidate tokens rather than reducing how many are emitted. If anything, a JSON envelope with punctuation, keys and quoting emits *more* tokens than the equivalent prose, and the injected system prompt adds input tokens as well. The saving is on the retry you no longer make, which is real and which shows up as variance reduction rather than as a lower mean.

Streaming is unchanged. Chapter 17's rules hold exactly: `text_delta` events arrive, `index` is the identity of a block, and the JSON is only a document after `message_stop`. Do not run `json.loads` on a partial accumulation.

23.29 Security Considerations

A schema is not a confidential channel. This is the sharpest security fact in the chapter and it is easy to miss. Prompts and responses are processed under zero data retention, but *the JSON schema itself is temporarily cached for up to 24 hours since last use*, separately from message content. The documentation is explicit that these cached schemas **do not receive the same PHI protections as prompts and responses**, and that PHI must not appear in schema **property names, enum values,** `const` **values or** `pattern` **regular expressions**. The same rule is repeated on the strict-tool-use page for `input_schema`. Structured outputs and strict tool use are HIPAA eligible; the schema is where that eligibility can be lost by a careless field name.

A description is prompt text. Field descriptions are rendered into the request and read by the model. A description assembled from user input is an injection surface with operator-adjacent placement, and Chapter 21's rule applies: untrusted content never goes anywhere that carries authority. Chapter 58 covers the attack in full.

A schema leaks your taxonomy. An enum enumerates your internal categories. If the schema is built from a customer-visible configuration, you have exported the taxonomy. That may be fine; it should be a decision.

The grammar constrains form, never authority. A schema-conformant response can still say `{"action": "refund", "amount": 100000}`. Chapter 21's thread — *the model proposes, your system disposes* — is completely unaffected by constrained decoding. Structured outputs make the proposal well-typed; they do not make it authorised. Every downstream authorisation check you had before the migration you still need after it.

Refusals still arrive as 200s. A schema does not change the refusal contract, and a pipeline that treats "parsed successfully" as "safe to act" now has a narrower failure window and the same failure.

23.30 Real Production Examples

A claims triage service. Moved from Chapter 22's envelope to `output_config.format`, and discovered on the first day that a required `patient_identifier` field name was flagged in review under the schema-caching rule. The field was renamed to `subject_ref` and the identifier moved into message content, where the retention guarantees apply. No code changed; the review caught it because the security checklist had a line for it.

A multi-tenant categoriser. Shipped per-tenant field descriptions as a personalisation feature. Grammar compilation metrics stayed flat, because they were correct: one grammar. The bill rose by roughly 170 per cent, and it took a fortnight to find because the team had a cache metric and it was the wrong cache.

An agentic scheduling assistant. Marked all fourteen of its tools `strict: true` and took a 400 on the twenty-fifth optional parameter, summed across tools that each looked trivial. The fix was not to split the request but to make eleven optional fields required with explicit defaults, which took the aggregate to 13 and also removed the required-first reordering that had been breaking a golden-file test.

A grounded research tool. Attempted the migration, hit the citations 400, and stopped. It still uses Chapter 22's envelope and its structural parser, and it is right to. The lesson its team wrote up internally is the one this chapter would choose to end on: **structured outputs make most of Chapter 22 unnecessary, and knowing exactly which part is still necessary is the whole skill.**

23.31 Certification Notes

Concept	Domain	Weight	What is examinable
Constrained decoding vs post-hoc validation	D4 Prompt Engineering & Structured Output	20 %	that the constraint is applied at sampling time and cannot be a validator
`output_config.format` vs `strict: true`	D4	20 %	which channel each constrains; that they combine
Supported JSON Schema subset	D4	20 %	`additionalProperties` must be `false`; the ten string formats; what is excluded
The three explicit limits	D4	20 %	20 / 24 / 16, and that the last two are combined across schemas
The four exceptions	D4, D5	20 %, 15 %	refusal, `max_tokens`, enum casing, property ordering
Incompatibilities	D4	20 %	citations returns 400; prefilling unsupported
Grammar cache vs prompt cache	D5 Context Management & Reliability	15 %	24 hours from last use; description changes invalidate one and not the other

Concept	Domain	Weight	What is examinable
Enum-tagged tool as the classification pattern	D2 Tool Design & MCP Integration	18 %	the documented replacement for prefill
Conformance vs correctness	D5	15 %	that they are separate measurements

CERTIFICATION NOTE

Exam tips

The examiners like the word *guarantee*. Structured outputs guarantee **schema compliance**, and the four documented exceptions are the highest-yield facts in this chapter. If a question describes a response that "completed normally with no error" and a value that is not quite in the enum, the answer is capitalisation.

Remember that 24 and 16 are **combined** limits across all strict schemas plus the JSON output schema in one request. A question that gives you four tools with six optional parameters each is testing exactly this.

`additionalProperties` is the most commonly missed detail: it is not optional, and `true` is not a legal value.

If citations appear in the same question as `output_config.format`, the answer is a 400.

And if a question asks what structured outputs do for accuracy, the answer is nothing.

23.32 Twenty Practice Questions

1. What is constrained decoding applied to?
2. Which request field enables JSON outputs?
3. Which property enables strict tool use, and where does it sit?
4. What two things does strict tool use guarantee?
5. What must `additionalProperties` be set to?
6. Name three unsupported JSON Schema keyword families.
7. How many string formats are supported?
8. What values may `minItems` take?
9. What is the limit on strict tools per request?
10. What is the limit on optional parameters, and across what scope?
11. What is the limit on union-typed parameters?
12. Which two `stop_reason` values mean the schema may not have been honoured?
13. What HTTP status accompanies a refusal?
14. What is not guaranteed about enum values?
15. In what order do properties appear in the output?
16. Which feature returns a 400 when combined with `output_config.format`?
17. How long are compiled grammars cached?

18. Which schema changes do *not* invalidate the grammar cache?
19. What does using structured outputs add to your input tokens?
20. Does constrained decoding improve accuracy?

23.33 Answers and Explanations

1. **The next-token distribution, at every decode step.** Not the finished string — that would be a validator. *Distractors:* "the response after generation" describes post-hoc validation; "the prompt" confuses it with the injected system prompt; "the tool result" is outside the grammar's scope entirely.
2. `output_config.format`**,** with `type: "json_schema"`. *Distractors:* `output_format` was the beta shape and still works for a transition period; `response_format` is another vendor's name; `format` alone is a sub-field.
3. `strict: true`**, as a top-level property of the tool definition,** beside `name`, `description` and `input_schema`. *Distractors:* inside `input_schema` is the commonest wrong answer; `tool_choice` forces a call but guarantees nothing about arguments.
4. **That** `input` **follows** `input_schema`**, and that** `name` **is always a valid tool name.** *Distractors:* it does not guarantee the tool is the *right* tool, nor that the call succeeds.
5. `false`**,** on every object, and it must be present. *Distractors:* `true` is explicitly unsupported; omitting it is a 400 on a raw schema and is silently patched by several SDKs.
6. **Numerical constraints, string length constraints, and array constraints beyond** `minItems` **0 or 1.** Also acceptable: recursion, complex types in enums, external `$ref`. *Distractor:* `enum` is supported; only complex *members* are not.
7. **Ten.** *Distractor:* any other count; the list is exact.
8. **0 and 1 only.** *Distractor:* "any non-negative integer" is the JSON Schema answer, not this one.
9. **20,** and non-strict tools do not count. *Distractor:* "20 tools in total" is wrong for the same reason.
10. **24, combined across all strict tool schemas and the JSON output schema in one request.** *Distractor:* "24 per tool" is the trap the documentation explicitly warns about.
11. **16.** These are `anyOf` or type arrays, and they carry exponential compilation cost.
12. `refusal` **and** `max_tokens`**.** *Distractors:* `tool_use` and `pause_turn` are non-terminal but the grammar still applied; `stop_sequence` is unaffected.
13. **200.** You are billed for the tokens generated, and `stop_details` carries the policy category.
14. **Capitalisation.** The response completes normally with no error and no special `stop_reason`. *Distractor:* "the value may be outside the enum entirely" overstates it; the drift is in casing.
15. **Required properties first in schema order, then optional properties in schema order.** *Distractor:* "declaration order" is the near-miss the documentation warns about.
16. **Citations.** They interleave citation blocks with text, which a strict grammar cannot express. *Distractor:* streaming and batch both work.
17. **24 hours from last use.** *Distractor:* 24 hours from creation; "from last use" is what keeps a warm schema warm.

18. **Changes to `name` or `description` only.** Structure changes and changes to the set of tools do invalidate it. *Distractor:* assuming the prompt cache behaves the same way costs 170.5 per cent in Section 23.22.
19. **An automatically injected system prompt describing the expected output format,** billed as ordinary input tokens. Its size is not published. *Distractor:* "nothing, the schema is free" is the belief this question exists to remove.
20. **No.** It guarantees the shape of the output and has no effect on whether the output is correct. In Section 23.19 the judgement ceiling is 15 of 24 with and without it.

23.34 Ten Practical Exercises

1. Convert one Chapter 22 envelope to a schema and keep the `<quotes>` channel as an array property.
2. Add a `notes` optional field to a schema, then predict the output key order and check it against the documented rule.
3. Write the enum-tagged classification tool that replaces prefill, and force it with `tool_choice: {"type": "any"}`.
4. Take a Pydantic model with `ge=100` and work out what actually goes on the wire.
5. Build a schema that trips "Schema is too complex for compilation" using only legal features, then reduce it by removing optionality alone.
6. Count the optional parameters across your entire tool set and compare with 24.
7. Write a test that a refusal fixture never reaches `json.loads`.
8. Measure your own enum drift rate over a week of production responses.
9. Price the difference between one schema and one-per-tenant at your volume.
10. Audit every property name, enum value and pattern in your schemas against the PHI rule.

23.35 Five Challenge Labs

1. **The correctness harness.** Build an evaluation set of 200 examples with a gold key, and report parse rate, conformance rate and correctness rate as three separate series over time. Show that only the third one moves when you change the prompt.
2. **The schema linter.** Extend `schema.py` to emit a machine-readable report and wire it into CI so a build fails on any UNKNOWN keyword or missing `additionalProperties`.
3. **The complexity budgeter.** Given a set of tools and a JSON output schema, report the aggregate against all three limits and suggest the cheapest single change that brings a failing set under.
4. **The dual-cache monitor.** Instrument a service to report grammar-compilation latency and prompt-cache write share on the same dashboard, and demonstrate a change that moves one and not the other.
5. **The citations fork.** Build one pipeline that answers with citations and Chapter 22's parser and one that answers with a schema, route by whether grounding is required, and measure the difference in both correctness and cost.

23.36 Ten Interview Questions

1. Explain constrained decoding to someone who thinks it is a validator.
2. Why can a schema not improve accuracy?
3. When would you use `strict: true` and *not* `output_config.format`?
4. A response completed normally and your enum lookup failed. What happened?
5. You are at 24 optional parameters. What do you change first, and why that?
6. Why is `additionalProperties: false` mandatory rather than recommended?
7. Which cache does a field description invalidate?
8. Your JSON parse errors went to zero and incidents did not fall. Explain.
9. What must never appear in a schema, and why is that different from a prompt?
10. When do you keep an XML envelope instead of migrating?

23.37 Summary

Chapter 22 ended with an envelope and a parser and the honest admission that the envelope was a form and not a promise. This chapter is the promise arriving. Structured outputs compile your JSON Schema into a grammar and apply it to the next-token distribution at every decode step, so that an invalid document is never generated rather than being generated and then rejected. The two features do different jobs on different channels — `output_config.format` constrains what Claude says, `strict: true` constrains how Claude calls you — and they combine in the same request, which in an agentic loop is the normal case.

The guarantee is real, and it is narrower than its headline. The supported slice of JSON Schema is defined and small: all the basic types, `enum` over scalars, `const`, `anyOf` and `allOf`, internal `$ref`, ten string formats, `minItems` of 0 or 1, and `additionalProperties` that must be present and must be `false`. Numerical and length constraints are out, recursion is out, external references are out, and several SDKs will quietly rewrite the ones that are out into prose in a description and validate them on your behalf afterwards — which is a good design and a reason to know which mode you are in. Complexity is capped three ways you can count, twenty strict tools and twenty-four optional parameters and sixteen unions, with the last two combined across every schema in the request, and capped again in ways you cannot see, because grammar size does not reduce to one dimension.

Four things the schema does not promise are worth more than everything else in the chapter. Two of them are `stop_reason` values you already branch on: a refusal arrives as an HTTP 200 with prose in the channel your code believes is JSON, and a `max_tokens` cut-off arrives as a fragment with no supported recovery. Two of them are silent: enum capitalisation is explicitly not guaranteed and produces no error and no special stop reason, and required properties are reordered ahead of optional ones in the output. The silent pair are the expensive pair, because nothing in your logs will tell you they happened.

The measurement at the centre of the chapter is the one to carry into every migration. On twenty-four BUILT tickets with a gold key, moving from prompt-and-parse to a schema took parse rate from 22 of 24 to 24 of 24 and conformance from 20 of 24 to 24 of 24, and delivered correctness from 50.0 per cent to 62.5 per cent — a real gain, earned entirely by rescuing answers that were right and were lost in transit. The judgement ceiling was 15 of 24 in both families. The grammar moved it by nothing, because it could

not: it constrains which tokens may be sampled and has no access to whether the sentence is true. A team that reads enum values with `==` gives the whole gain back, dropping to 12 of 24, on three responses the model got right. And forty schemas that differ only in a description string compile to exactly one grammar while taking the prompt-cache hit rate from 98.3 per cent to 47.1 per cent and the monthly bill from $2,069.23 to $5,596.92, because the two caches have different keys and the change that is free for one is ruinous for the other.

What remains of Chapter 22 is now precisely delimited, which was this chapter's job. Delimiter collision on the output side is gone. The one-line regex is gone. The structural parser is gone from every path where a grammar runs — and recovers 0 of 2 on the paths where one does not, which is a negative result worth publishing, because it says the answer there is control flow rather than a cleverer reader. The place the envelope genuinely survives is the citations fork: grounding and JSON outputs return a 400 together, so a cited-answer pipeline stays exactly where Chapter 22 left it. Knowing which of your pipelines that is, before you migrate any of them, is the whole skill.

KEY TAKEAWAYS

1. **Constrained decoding edits the distribution, not the string.** The grammar is applied at every decode step, so an invalid document is never produced; there is nothing to retry and nothing to repair.
2. **Two features, two channels.** `output_config.format` constrains the text Claude emits; `strict: true` constrains the `input` of a tool call. They combine, and grammar state resets between sections so thinking stays free.
3. **The supported subset is small and exact.** `additionalProperties` must be present and `false`; ten string formats; `minItems` of 0 or 1 only; no numerical or length constraints; no recursion; no external `$ref`.
4. **A keyword documented neither way is UNKNOWN and must raise.** `oneOf` is the case that will find you.
5. **Twenty, twenty-four, sixteen — and the last two are combined** across every strict schema and the JSON output schema in a single request.
6. **Two of the seven `stop_reason` values mean no grammar was enforced.** Branch before you parse; a refusal is an HTTP 200 and a truncation has no supported recovery.
7. **Enum capitalisation is not guaranteed and fails silently.** Compare case-insensitively, canonicalise on read, and never declare two members differing only in case.
8. **Required properties are emitted before optional ones,** regardless of declaration order.
9. **Citations and JSON outputs return a 400 together.** That fork keeps Chapter 22 alive, permanently and by design.
10. **A schema constrains the shape of an answer and says nothing about its truth.** Conformance and correctness are different measurements; report both, or you have stopped measuring the model.

23.38 Glossary

Term	Definition
Constrained decoding	Restricting the next-token distribution at each decode step so only grammar-legal tokens can be sampled.

Term	Definition
Grammar-constrained sampling	The documentation's term for the same mechanism applied to strict tool use.
Structured outputs	The umbrella feature comprising JSON outputs and strict tool use.
JSON outputs	`output_config.format` with `type: "json_schema"`; constrains Claude's text response.
Strict tool use	`strict: true` on a tool definition; constrains the tool's `input` and guarantees a valid tool `name`.
`output_config`	The top-level request field carrying `format`, `effort` and `task_budget`.
Grammar cache	The 24-hour-from-last-use cache of compiled schemas, keyed on schema structure and tool set.
Prompt cache	Chapter 11's prefix cache, keyed on the byte-exact request prefix.
Optional parameter	A property not listed in `required`; capped at 24 combined across all strict schemas in a request.
Union-typed parameter	A property using `anyOf` or a type array; capped at 16, combined.
`additionalProperties`	The JSON Schema keyword controlling undeclared fields; must be present and `false`.
Enum drift	An enum value returned with capitalisation that differs from the declared member; documented and silent.
Property ordering	The documented rule that required properties are emitted first, then optional ones.
Conformance	Whether a response satisfies the schema.
Correctness	Whether a response is the right answer. Independent of conformance.
Judgement ceiling	The correctness rate with formatting failures removed; the number a grammar cannot move.
Schema transformation	The SDK behaviour of stripping unsupported constraints into descriptions and validating client-side.
Contract	In this chapter's code, the single object owning a schema and emitting both request shapes.

23.39 Further Reading

Primary sources, all read on 1 August 2026. `platform.claude.com/docs/en/build-with-claude/structured-outputs` is the authority for the supported subset, the complexity limits, the four exceptions, the caching behaviour and the per-platform availability lists. `platform.claude.com/docs/en/agents-and-tools/tool-use/strict-tool-use` covers `strict: true`, the guarantees it makes and the data-retention rules for `input_schema`. `platform.claude.com/docs/en/agents-and-tools/tool-use/define-tools` gives the tool definition shape, the `^[a-zA-Z0-9_-]{1,64}$` name constraint, `input_examples`, and the `tool_choice` options —

including the note that combining `tool_choice: {"type": "any"}` with strict tools guarantees both that a tool is called and that its arguments are valid. `json-schema.org` is the specification the subset is drawn from; read it knowing that most of it is not supported here.

Where to go next in this book. Chapter 24 turns the correctness number of Section 23.19 into a discipline: prompt evaluation and regression suites, including how large an evaluation set must be before a difference in correctness is a finding rather than a rounding error — Chapter 12's sample-size arithmetic applied to prompts. **Chapter 25** covers guardrails, validation and prompt debugging, and is where the authorisation layer that a schema does not provide gets built properly. **Chapters 31 to 35** are tool use in the round: schema design as an affordance, naming, error contracts and idempotency, with this chapter's `strict: true` as one tool among several. **Chapter 58** takes up schema descriptions as an injection surface. And **Chapter 22** stays on the shelf, because the citations fork means you will need it again.